

Predicción de clientes con riesgo de compensación por calidad del servicio en un operador de red eléctrica

Autores: María Delgado, Leonardo Álvarez, Cesar Vásquez y Fabian Lasso

1. Resumen del problema abordado

El proyecto busca anticipar la ocurrencia de compensaciones económicas a clientes del servicio eléctrico, derivadas de deficiencias en la calidad del suministro. Actualmente, estas compensaciones se determinan de forma reactiva, es decir, después de que la interrupción ha ocurrido durante un período prolongado o varias veces, lo que impide que la empresa tome medidas preventivas. Este fenómeno tiene un impacto directo sobre los gastos operativos, la satisfacción del usuario y el cumplimiento de los estándares regulatorios establecidos por los entes de control.

En este contexto, disponer de una herramienta predictiva permitiría identificar con anticipación los clientes con mayor riesgo de ser compensados, posibilitando priorizar mantenimientos, focalizar inversiones y mejorar la gestión operativa. Así, la solución propuesta transforma el proceso de control de calidad del servicio en un enfoque proactivo y basado en datos, aportando valor tanto estratégico como financiero.

1.1 Pregunta de negocio y alcance del proyecto: ¿Es posible anticipar qué clientes tienen mayor probabilidad de generar compensaciones por baja calidad del servicio eléctrico, utilizando modelos predictivos basados en registros históricos y variables operativas del sistema?

El alcance abarca el desarrollo de una solución analítica integral que comprende desde la recolección, integración y limpieza de los datos hasta la construcción, validación y despliegue del modelo predictivo en la nube. Para ello, se implementará una arquitectura en Amazon Web Services (AWS), apoyada en herramientas como DVC, MLflow y GitHub, además de servicios y flujos que permitan la integración y el despliegue continuo (CI/CD) para garantizar trazabilidad, automatización y eficiencia en todo el ciclo de vida del modelo. El resultado esperado es un prototipo funcional que predice con precisión el riesgo de compensación y una visualización interactiva que explore las áreas con mayor frecuencia de interrupciones, permitiendo obtener conocimientos basados en datos para la toma de decisiones.

1.2 Descripción de los conjuntos de datos: Se utilizarán dos bases principales en formato CSV, integradas mediante la variable NIU (identificador único del cliente):

Base de clientes: contiene información técnica, geográfica y operativa del punto de suministro, incluyendo tipo y nivel de conexión, ubicación, estrato, y coordenadas geográficas (latitud, longitud, altitud).

Base de indicadores de calidad y compensaciones: registra variables históricas de continuidad del servicio (duración, frecuencia de interrupciones), valores de compensación económica, consumo mensual y datos regulatorios asociados a la calidad.

A partir de la combinación y limpieza de estas dos fuentes, se realizó un análisis exploratorio, se seleccionaron características relevantes y se creó un conjunto de datos analíticos unificado para construir el modelo predictivo y la visualización interactiva

2. Desarrollo y Evaluación de Modelos

Para la construcción se seleccionaron variables numéricas y categóricas de la información técnica del servicio y los registros comerciales para construir modelos predictivos. Estas variables fueron agregadas, limpiadas y preprocesadas en dos etapas: primero, las variables categóricas se convirtieron en indicadores numéricos, y luego se estandarizaron las variables continuas para que operaran en una escala similar. Dado que el número de compensaciones era considerablemente menor que el de no compensaciones, se utilizó una estrategia de reequilibrio para los datos de entrenamiento, eliminando así el potencial sesgo de los modelos hacia la clase mayoritaria. El conjunto de datos, tras el procesamiento, se dividió en dos: uno para entrenar los modelos y otro para medir el rendimiento real sin haber sido utilizada durante el aprendizaje.

2.1 Experimentos y Selección

Para asegurar la trazabilidad de los experimentos, se utilizó MLflow para evaluar tres algoritmos. Se describieron diferentes escenarios de configuración en cada modelo y se parametrizaron internamente, identificando aquellos que proporcionaban el mejor rendimiento. En todas las aplicaciones, el criterio principal de evaluación fue el AUC-ROC, complementado con métricas de precisión, exactitud, recall y F1-score, calculadas en el conjunto de datos de prueba. Los resultados obtenidos, junto con la configuración seleccionada para cada modelo, se presentan a continuación:

Tabla 1. Resumen de Desempeño de Modelos (Conjunto de Prueba)

Modelo	Mejor configuración encontrada	AUC	Exactitud	Recall (Clase Compensada)	Precisión clase compensada
Regresión Logística	Regularización (penalty): L2; intensidad de regularización (C): 0.1; método de optimización (solver): saga; balance interno de clases (class_weight = balanced): activado	0.97	0.88	0.94	0.75
Random Forest (Seleccionado)	número de árboles (n_estimators): 300; profundidad máxima (max_depth): 20; proporción mínima para división(min_samples_split): 0.01; proporción mínima por hoja (min_samples_leaf): 0.005	0.98	0.90	0.97	0.79
LDA	Método de estimación de covarianzas (solver = svd): estándar; prior automático (priors = None)	0.67	0.68	0.32	0.54

El modelo **Random Forest** fue seleccionado por presentar la mayor estabilidad y capacidad de generalización, el modelo LDA mostró desempeño considerablemente inferior frente a los anteriores, y el modelo de regresión logística presento el segundo mejor desempeño.

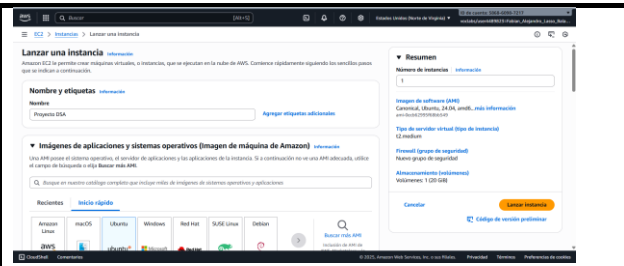
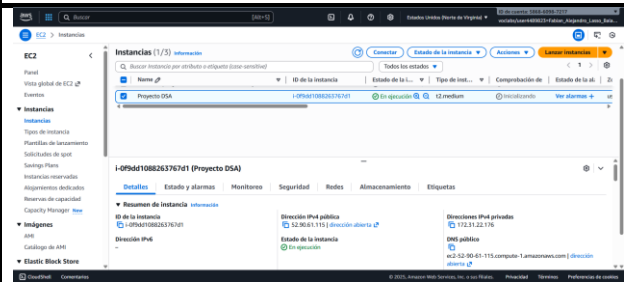
Selección del modelo: De acuerdo con los resultados obtenidos sobre la muestra de prueba, el modelo seleccionado para la siguiente fase del proyecto fue el Random Forest, debido a que presentó las métricas más altas y mayor estabilidad al momento de identificar correctamente a los clientes con riesgo de compensación.

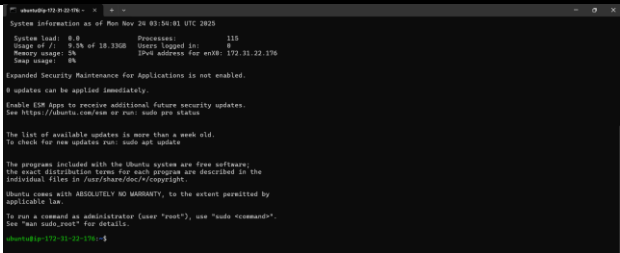
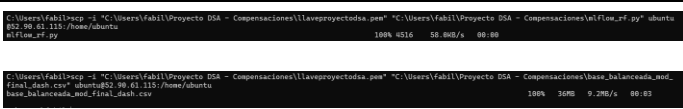
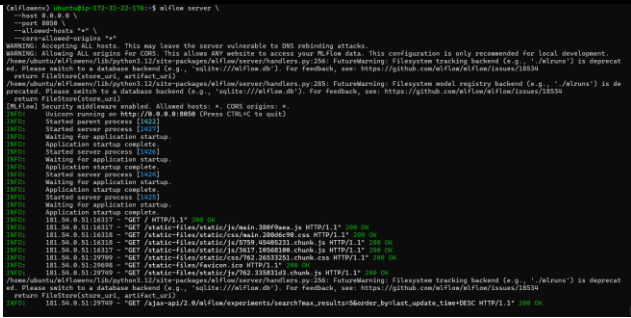
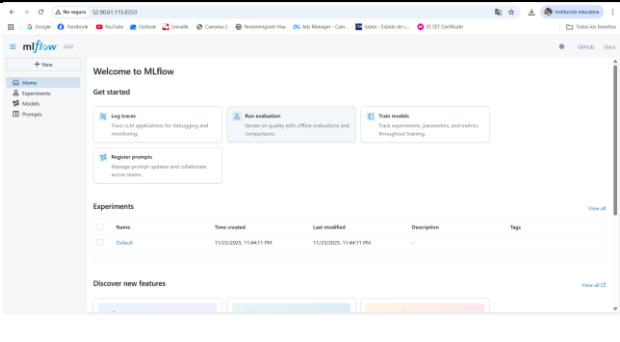
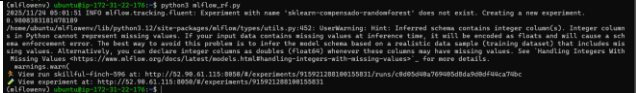
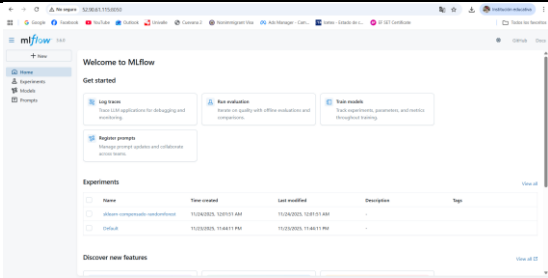
3.1 Despliegue de Infraestructura y Trazabilidad Experimental en AWS

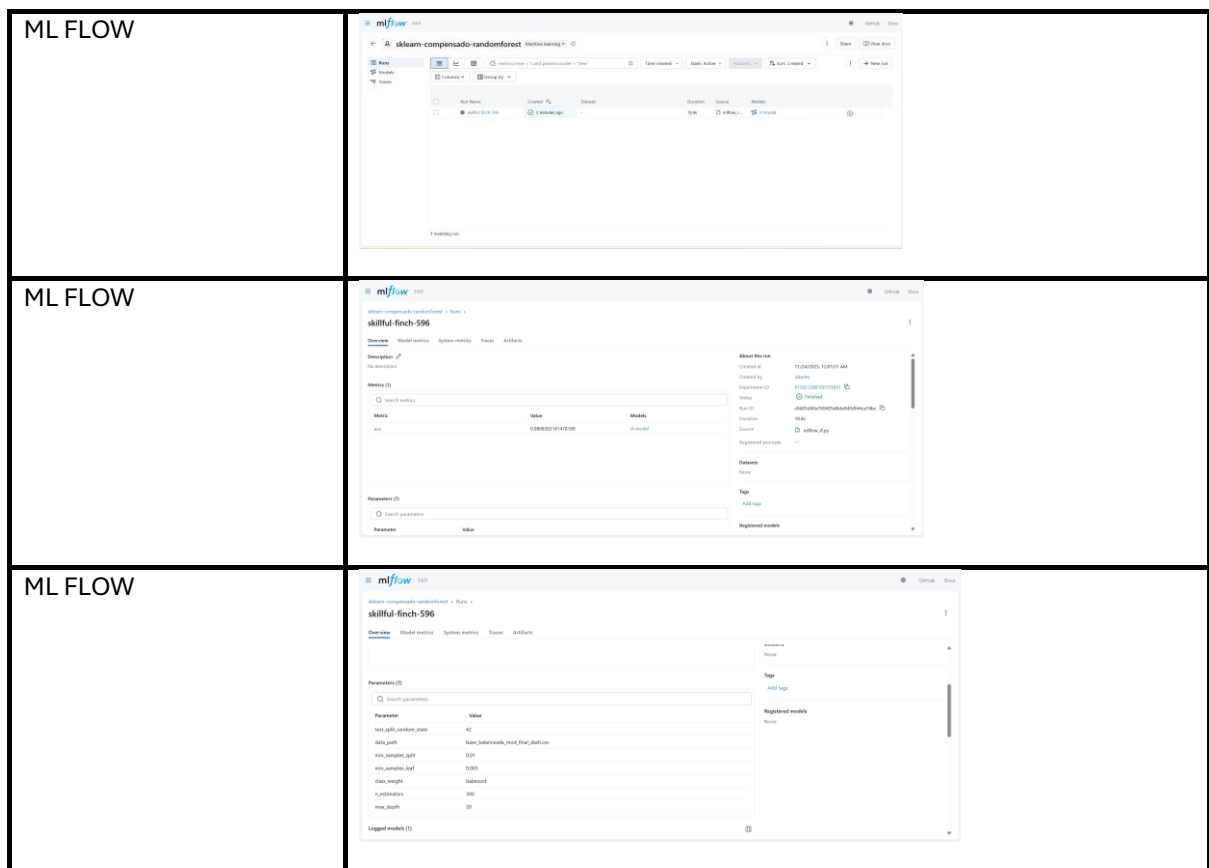
El proceso inició con el lanzamiento y aprovisionamiento de la infraestructura en la nube (IaaS) a través de la consola de AWS EC2 en una instancia de servidor Ubuntu con un tipo de servidor t2.medium necesario para proporcionar los recursos de computación requeridos.

Según el panel de gestión, el despliegue fue correcto, la instancia "Proyecto DSA" se lanzó con éxito y tenemos una instancia en funcionamiento con IP pública 52.90.61.115. Después de que el proyecto estuviera en línea, se estableció una conexión SSH segura a través de las claves privadas (.pem) para iniciar sesión en el terminal del servidor de forma remota. Con la infraestructura en marcha, los activos para el proyecto se transfieren desde el entorno local utilizando el protocolo SCP y el código fuente está intacto (mlflow_rf.py) y el conjunto de datos procesados (base_balanceada_mod_final_dash.csv). Luego, se creó el entorno virtual de Python y se desplegó el servidor de seguimiento de MLflow, de modo que el servidor estaba abierto en el puerto 8050 host 0.0.0.0 y nos permitió con éxito obtener interfaces gráficas web en el navegador para monitorear experimentos.

Finalmente, se ejecutó el pipeline de entrenamiento del modelo Random Forest, logrando una trazabilidad completa del ciclo de vida del modelo. Las capturas de pantalla adjuntas a continuación confirman que se registra automáticamente (en MLflow) en el experimento sklearn-compensado-randomforest, donde se registraron los hiperparámetros de configuración (por ejemplo, n_estimators=300, class_weight=balanced) y las métricas de evaluación resultantes, produciendo un AUC ~0.98, demostrando la adecuada incorporación y operación de la solución analítica en el entorno de la nube. La evidencia del proceso se mostró en la tabla a continuación:

Lanzamiento de instancia	
Especificaciones de la instancia	

Se realizo la conexión a la instancia	
Se enviaron los archivos necesarios a la instancia	
Se ejecuto el MLflow y se habilito el puerto correspondiente	
Se valida el funcionamiento del mlflow	
Luego se corrió el primer experimento del nuevo modelo	
Evidencia creación del modelo y primer experimento	



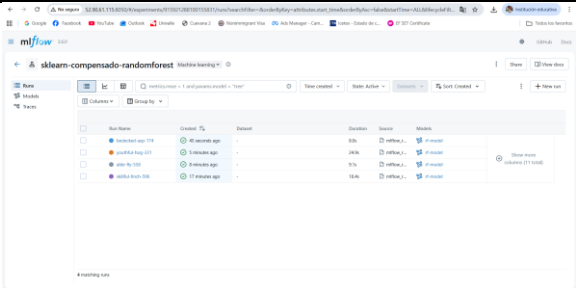
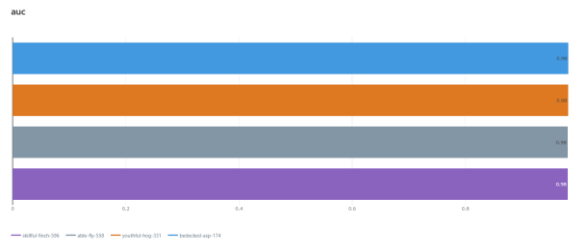
3.2 Iteración de Hiperparámetros y Empaquetamiento Reproducible del Modelo

Con la línea base establecida, se procedió a una fase de experimentación iterativa para optimizar el modelo, buscando un equilibrio entre precisión y costo computacional. Los hiperparámetros fueron ajustados en el código fuente, simplificando la complejidad del estimador al establecer `n_estimators` en 120 y `max_depth` en 10, además de registrar automáticamente nuevas ejecuciones en MLflow. Esta plataforma no solo permitió almacenar el historial de ejecuciones (como en las ejecuciones "skillful-finch" y "able-fly"), sino que también facilitó la comparación visual directa de la métrica AUC entre ambas versiones, lo que contribuyó a la selección del modelo final basado en evidencia objetiva.

Con el modelo elegido, el enfoque del flujo de trabajo se centró en la ingeniería de productos de datos, denominado "Empaquetado del Modelo". El repositorio estructurado (Package_Final-crf.zip) fue trasladado a una instancia EC2, donde se automatizó el proceso utilizando Tox. Como se evidenció en las capturas de consola, el comando `tox run -e train` se ejecutó con éxito, permitiendo la instalación de las dependencias en un entorno aislado y la ejecución completa de la tubería de entrenamiento.

Posteriormente, se generaron los artefactos de distribución en la carpeta `dist`, incluyendo el archivo instalable (`wheel/.whl`) y el código fuente comprimido. Se creó un directorio de prueba en el servidor para verificar la fiabilidad del paquete antes de su lanzamiento; el archivo `.whl` fue copiado junto con el script de validación (`test-package.py`). Al ejecutar este script, el modelo empaquetado generó datos predichos exitosamente (etiquetados en matrices binarias en la consola), lo que demuestra su capacidad de inferencia.

El paquete validado se descargó posteriormente al entorno local mediante el protocolo SCP y se subió al repositorio de GitHub. Los artefactos fueron versionados y están disponibles para su despliegue en producción, asegurando así que el modelo sea reproducible y fácil de desplegar. A continuación, las evidencias:

modificaron los parámetros	<pre> n_estimators = 120 max_depth = 10 min_samples_split = 0.02 min_samples_leaf = 0.01 class_weight = "balanced" </pre>	
Luego se ejecutó de nuevo con los nuevos parametros	<pre> (mf10env) ubuntu@172-31-22-176:~\$ python mf10v.py 0.9887881917843812 /home/ubuntu/.local/lib/python3.12/site-packages/mlflow/types/utils.py:83: UserWarning: Hint: Inferred schema contains integer column(s). Integer column s in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a sch ema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes mis sing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See "Handling Integers With Missing Values" <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values> for more details. warnings.warn(^ View run able-fiy-358 at: http://122.90.61.115:8080/#/experiments/915921288160155831/runs/7204d81a78e7088f95591873358b67ff ^ View experiment at: http://122.90.61.115:8080/#/experiments/915921288160155831 (mf10env) ubuntu@172-31-22-176:~\$ </pre>	
Evidencia ejecuciones		
Gráfico del AUC de los experimentos realizado con el modelo RF		
Se genera el archivo.zip para el empaquetamiento y se envia a la instancia	<pre> C:\Users\adall\Proyecto DSA - Compensaciones\lavanproyectada.py" "C:\Users\adall\Proyecto DSA - Compensaciones\Package_Final-crf-11 ^ ubuntu@172-31-22-176:~\$ mv Package_Final-crf-11 Package_Final-crf-11.zip C:\Users\adall> </pre>	
Evidencia archivo en la instancia	<pre> ubuntu@172-31-22-176:~\$ ls -l total 42796 -rw-rw-r-- 1 ubuntu ubuntu 6202117 Nov 24 23:28 Package_Final-crf.zip -rw-rw-r-- 1 ubuntu ubuntu 37595818 Nov 24 04:39 base_balanceada_mod_final_dash.csv drwxrwxr-x 3 ubuntu ubuntu 4896 Nov 24 05:02 artifacts -rw-rw-r-- 1 ubuntu ubuntu 4587 Nov 24 05:18 mf10v.py drwxrwxr-x 6 ubuntu ubuntu 4896 Nov 24 04:18 mlflowenv drwxrwxr-x 6 ubuntu ubuntu 4896 Nov 24 05:01 mlruns ubuntu@172-31-22-176:~\$ </pre>	
Se intalaron los requerimientos necesarios, se descomprimió y luego se ingresó la carpeta y se ejecutó el tox	<pre> (env-tox) ubuntu@172-31-22-176:~\$ which tox /home/ubuntu/.env/bin/tox (env-tox) ubuntu@172-31-22-176:~\$ cd Package_Final-crf (env-tox) ubuntu@172-31-22-176:~/Package_Final-crf\$ tox run -e train train: commands[0]: python model/train_pipeline.py train: OK (28.40 seconds) completions: 1 (38.40 seconds) (env-tox) ubuntu@172-31-22-176:~/Package_Final-crf\$ (env-tox) ubuntu@172-31-22-176:~\$ which tox /home/ubuntu/.env/bin/tox (env-tox) ubuntu@172-31-22-176:~\$ cd Package_Final-crf (env-tox) ubuntu@172-31-22-176:~/Package_Final-crf\$ tox run -e train train: commands[0]: python model/train_pipeline.py train: OK (28.40 seconds) completions: 1 (38.40 seconds) (env-tox) ubuntu@172-31-22-176:~/Package_Final-crf\$ tox run -e test_package test_package: remove tox env folder /home/ubuntu/Package_Final-crf/tox/test_package test_package: Install deps: pip install -r /home/ubuntu/Package_Final-crf/requirements/test_requirements.txt test_package: commands[0]: python model/train_pipeline.py test_package: commands[1]: pytest -s -vv tests/ platform linux -- Python 3.12.3, pytest-7.4.0, pluggy-1.6.0 -- /home/ubuntu/Package_Final-crf/tox/test_package/bin/python cachedir: /home/ubuntu/Package_Final-crf/tox/test_package/.pytest_cache rootdir: /home/ubuntu/Package_Final-crf/tox/test_package collected 1 item tests/test_prediction.py::test_mdr_prediction PASSED tests/test_prediction.py: test_mdr_prediction PASSED test_package: OK (40.80 seconds) completions: 1 (50.80 seconds) (env-tox) ubuntu@172-31-22-176:~/Package_Final-crf\$ </pre>	
Creación del paquete	<pre> adding 'model/config/_init_.py' adding 'model/config/core.py' adding 'model/datasets/_init_.py' adding 'model/processing/_init_.py' adding 'model/processing/data_manager.py' adding 'model/processing/features.py' adding 'model/processing/validation.py' adding 'model/trained/_init_.py' adding 'model/trained/model-compensaciones-output8.0.1.pkl' adding 'modelo_compensaciones-0.0.1.dist-info/METADATA' adding 'modelo_compensaciones-0.0.1.dist-info/README' adding 'modelo_compensaciones-0.0.1.dist-info/top_level.txt' adding 'modelo_compensaciones-0.0.1.dist-info/RECORD' removing build/bdist.linux-x86_64/mhwhl Successfully built modelo_compensaciones-0.0.1.tar.gz and modelo_compensaciones-0.0.1-py3-none-any.whl (env-tox) ubuntu@172-31-22-176:~/Package_Final-crf\$ </pre>	

Se creo la carpeta test, se copió ahí el archivo .whl y también se subieron los archivos de test del paquete y la base para probarla

Se probó el paquete

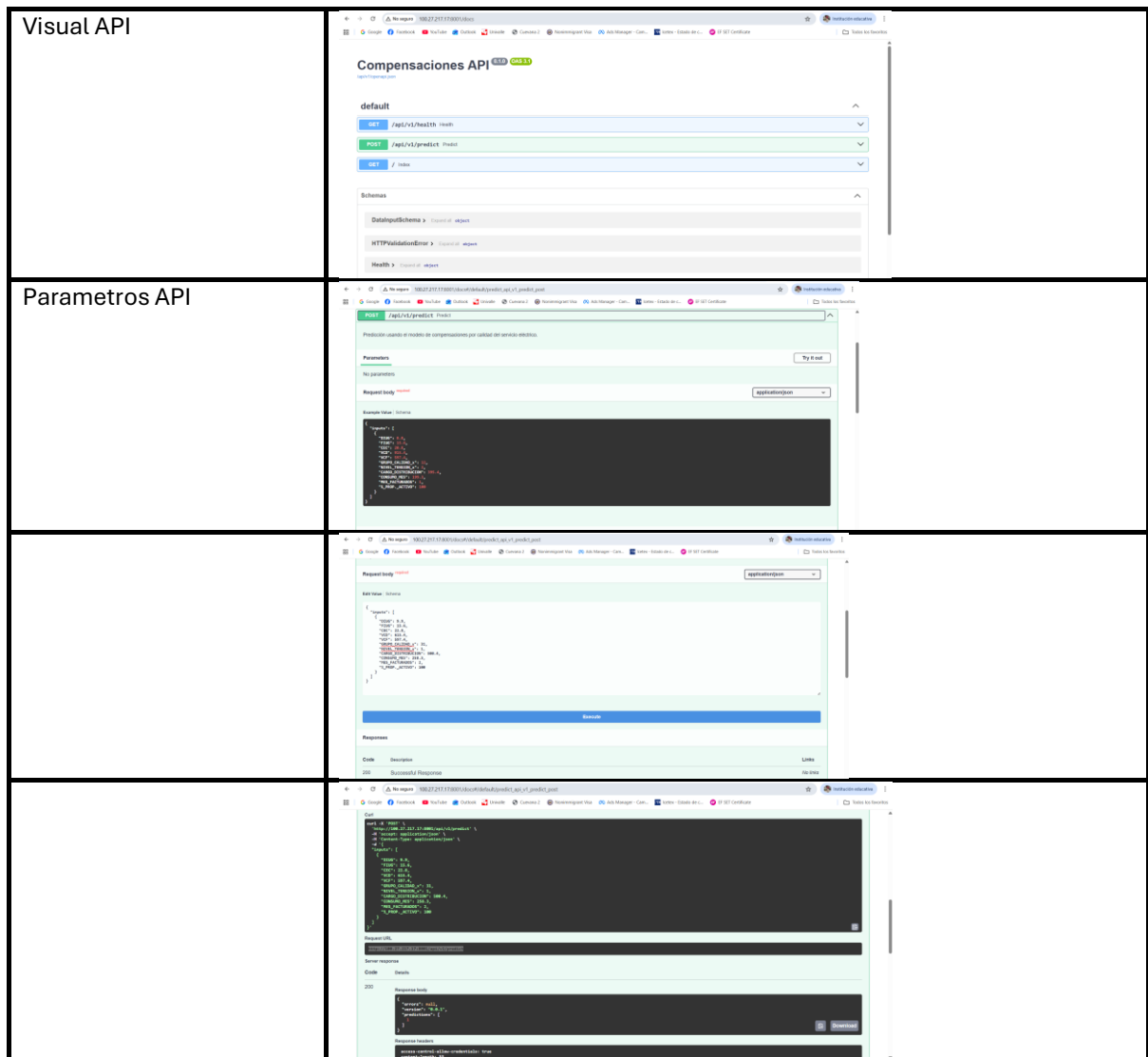
Y finalmente se descargó el paquete en el equipo local

Se guardó en el equipo y se subió al repositorio

4.1 Despliegue de la API

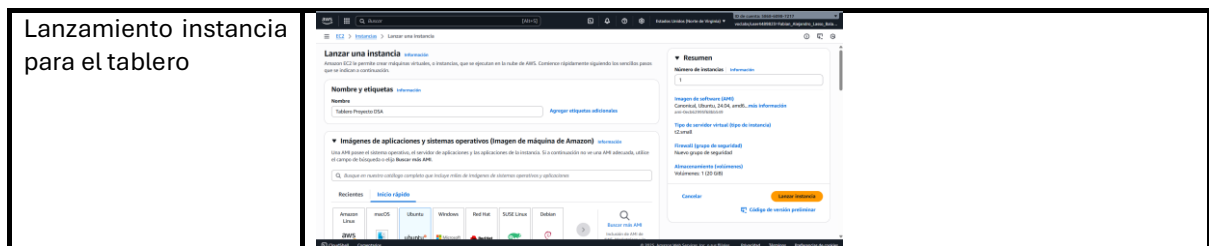
Luego de abrir el puerto correspondiente en ec2 de aws,. El despliegue de la interfaz de programación (API) inició con la transferencia segura de los artefactos del proyecto (Compensaciones-API.zip) al servidor EC2 mediante el protocolo SCP. Tras la descompresión en el entorno remoto, se ejecutó una batería de pruebas automatizadas utilizando **Tox** (`tox run -e test_app`), las cuales validaron la integridad de los endpoints antes de la puesta en marcha. Con la confirmación de las pruebas, se procedió a levantar el servicio utilizando **Uvicorn** en el puerto 8001, habilitando el acceso a la documentación interactiva (Swagger UI). Finalmente, se verificó la funcionalidad del sistema realizando una petición **POST** al endpoint `/predict`; al ingresar los parámetros operativos de un cliente (formato JSON), la API procesó la solicitud y retornó una predicción exitosa (código 200), confirmando que el modelo está disponible para realizar inferencias en tiempo real mediante la API.

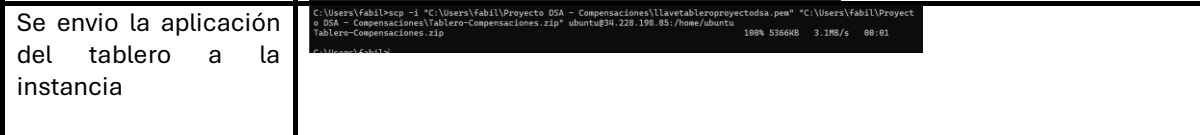
[illegible]



4.2 Despliegue del tablero

Para visualizar los resultados, configuramos una instancia específica asignada a AWS EC2 (tipo t2.small con Ubuntu) y la nombramos 'DSA Project Dashboard'. El despliegue se realizó transfiriendo archivos de solución (Dashboard-Compensations.zip) a través del protocolo SCP y ejecutando el script principal (python3 app.py), lo que inició efectivamente el servidor web en el puerto 8050. Al acceder a la interfaz a través de la IP pública, se verificó el correcto funcionamiento de dos componentes clave: un módulo de inteligencia geoespacial que mapea áreas de riesgo y KPIs de negocio, y un simulador de predicción interactivo. Este último fue validado ingresando manualmente variables operativas (por ejemplo, DIUG y FIUG), obteniendo una respuesta de clasificación de riesgo ('COMPENSADO (1)') para comprobar si el frontend del dashboard y la lógica del modelo predictivo se han integrado correctamente. Evidencias:



[illegible]

Validación del funcionamiento del tablero obteniendo las predicciones desde la api desplegada

Visuales del tablero

The dashboard displays three visualizations of compensation prediction results. Each visualization includes a line chart and a simulation form.

Visualización 1 (Top):

- Gráfico:** Muestra el control (Y-axis, 0 a 100,000) frente al periodo (X-axis, 2023-07 a 2023-09). La línea de control aumenta linealmente.
- Simulador de predicción de compensación:**
 - INGRESA LOS VALORES DE LAS VARIABLES CLAVE PARA ENVIAR UNA SOLICITUD A LA API DE COMPENSACIONES Y OBTENER LA PROBABILIDAD / CLASIFICACIÓN.
 - Variables de entrada: DUG (10.0), FUG (10.0), CEC (10.0), VCD (10.0), VOF (10.0), CARGO_DISTRIBUCION (10.0), GRUPO_CALIDAD_x (10), NIVEL_TENSION_x (10), CONSUMO_MES (10.0), MES_FACTURADOS (10), %PROP_ACTIVOS (100).
 - Botón: **Calcular predicción**
 - Predicción modelo: 1.000

Visualización 2 (Middle):

- Gráfico:** Muestra el control (Y-axis, 0 a 100,000) frente al periodo (X-axis, 2023-07 a 2023-09). La línea de control aumenta linealmente.
- Simulador de predicción de compensación:**
 - INGRESA LOS VALORES DE LAS VARIABLES CLAVE PARA ENVIAR UNA SOLICITUD A LA API DE COMPENSACIONES Y OBTENER LA PROBABILIDAD / CLASIFICACIÓN.
 - Variables de entrada: DUG (10.0), FUG (10.0), CEC (10.0), VCD (10.0), VOF (10.0), CARGO_DISTRIBUCION (10.0), GRUPO_CALIDAD_x (10), NIVEL_TENSION_x (10), CONSUMO_MES (10.0), MES_FACTURADOS (10), %PROP_ACTIVOS (100).
 - Botón: **Calcular predicción**
 - Predicción modelo: 1.000

Visualización 3 (Bottom):

- Gráfico:** Muestra el control (Y-axis, 0 a 100,000) frente al periodo (X-axis, 2023-07 a 2023-09). La línea de control aumenta linealmente.
- Simulador de predicción de compensación:**
 - INGRESA LOS VALORES DE LAS VARIABLES CLAVE PARA ENVIAR UNA SOLICITUD A LA API DE COMPENSACIONES Y OBTENER LA PROBABILIDAD / CLASIFICACIÓN.
 - Variables de entrada: DUG (10.0), FUG (10.0), CEC (10.0), VCD (10.0), VOF (10.0), CARGO_DISTRIBUCION (10.0), GRUPO_CALIDAD_x (10), NIVEL_TENSION_x (10), CONSUMO_MES (10.0), MES_FACTURADOS (10), %PROP_ACTIVOS (100).
 - Botón: **Calcular predicción**
 - Predicción modelo: 1.000

Predicciones presentadas en tablero



5. Principales Resultados y Conclusiones

El desarrollo de este proyecto de soluciones analítica ha permitido transformar el enfoque de gestión de calidad del servicio eléctrico, pasando de un esquema reactivo a brindar herramientas para tener uno preventivo basado en datos. La integración de código fuente, pipelines de entrenamiento y empaquetado se encuentra versionada y disponible para colaboración en el siguiente repositorio: <https://github.com/Fabian-11/Proyecto---Despliegue-de-Soluciones-Analiticas>

Conclusiones.

1. Se logro un modelo campeón con buena capacidad predictiva: El modelo final (Random Forest) dió un AUC de 0.98 y, también, un Recall del 97% para la clase compensada. En términos de negocio, esto significa que la herramienta es capaz de detectar a una gran cantidad de los clientes que van a requerir compensación, minimizando los "falsos negativos" y permitiendo a la compañía actuar antes de incurrir en penalizaciones económicas
2. Robustez del despliegue y disponibilidad: El uso combinado de las herramientas vistas en el curso (ASW EC2, GIT, MLFLOW, TABLEROS), permitió una arquitectura escalable en la nube, siendo posible así construir un flujo automatizado de bajo costo y alta fiabilidad, la solución es producto de datos empaquetado, reproducible y listo para integrarse en los sistemas operativos de la empresa.
3. Valor estratégico y toma de decisiones: La visualización de los resultados a través del tablero interactivo (desplegado) y el mapa georreferenciado cierra la brecha entre la ciencia de datos y la operación. Al identificar "zonas calientes" con alta probabilidad de fallo, los stakeholders pueden priorizar cuadrillas de mantenimiento y focalizar inversiones de manera eficiente, según consideren dado su criterio de experto a partir de los hallazgos compartidos por el equipo, dando espacio a potenciales impactos directos en la reducción de gastos operativos y la mejora de la satisfacción del usuario final.